A 10T1C SRAM-Based Computing-in-Memory Unit Supporting Logic and MAC Computing for Energy-Efficient Edge AI Chips

Zhiting Lin
School of Integrated Circuits
Anhui University
Hefei, China
ztlin@ahu.edu.cn

Chenglong Duan
School of Integrated Circuits
Anhui University
Hefei, China
1161798808@qq.com

Miao Long
School of Integrated Circuits
Anhui University
Hefei, China
504345904@qq.com

Juntao Ge
School of Integrated Circuits
Anhui University
Hefei, China
2168771181@qq.com

Fugui Jiang
School of Integrated Circuits
Anhui University
Hefei, China
1718721138@qq.com

Yang Yang
School of Integrated Circuits
Anhui University
Hefei, China
1278657916@qq.com

Xiulong Wu*
School of Integrated Circuits
Anhui University
Hefei, China
xiulong@ahu.edu.cn

2025 10th International Conference on Integrated Circuits and Microsystems (ICICM) | 979-8-3315-8849-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICICM66614.2025.11315968

***Abstract*—This paper proposes an innovative compute-in-memory (CIM) integrated circuit based on a 10T1C SRAM. The design augments a standard 6T-SRAM cell with four NMOS transistors and one capacitor, enabling simultaneous data storage and in-memory AND/XNOR logic operations. The 10T1C cell supports three operating modes: data storage, in-memory AND, and in-memory XNOR. Furthermore, arrayed configurations enable more complex computations, including multiply-accumulate (MAC) operations and binary neural network (BNN) inference. Simulated in a 28-nm CMOS process, the core achieves an energy efficiency of 18.3–41.8 TOPS/W for Boolean operations over a 0.7–0.9 V supply range. This high efficiency makes it well suited for energy-constrained edge-AI applications that require both storage and logic capabilities.**

Keywords— CIM, SRAM, Boolean logic, Edge AI, CNN.

I. INTRODUCTION

The rapid growth of artificial intelligence (AI) workloads on edge devices demands highly efficient computing architectures that tightly integrate memory and processing. Traditional von Neumann architectures are fundamentally constrained by the well-known "memory wall" [1], [2], which arises from the physical separation of memory and compute units. This separation forces frequent, energy-intensive data transfers between memory and processors, causing substantial energy overhead and latency for data-intensive edge-AI tasks. Compute-in-memory (CIM) technology has emerged as a promising solution to this challenge [3], [4]. By embedding computation directly within memory arrays, CIM greatly reduces data movement and thereby offers significant gains in both energy efficiency and processing speed.

Although SRAM-based CIM designs have made progress for key arithmetic tasks such as multiply-accumulate (MAC) operations [5], [6], efficiently implementing Boolean logic operations in memory remains a challenge. Boolean functions like AND and XNOR are fundamental to many AI algorithms, especially in binary neural networks (BNNs) where operations are binarized to improve efficiency. Existing CIM designs often trade off storage density against computational capability, and few integrate robust memory and versatile in-memory logic in a single cell without sacrificing area or energy [7], [8].

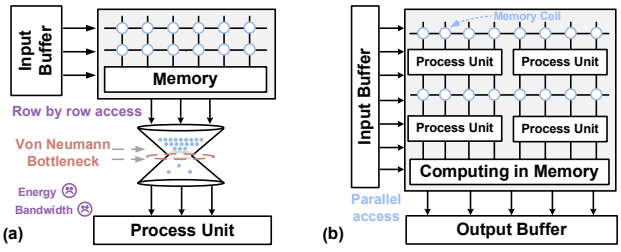


Fig. 1. (a) Description of Von Neumann architecture. (b) Description of Computing-in-memory architecture.

This paper makes three key contributions:

1. A 10T1C-SRAM cell based on the standard 6T-SRAM structure is proposed, which extends the functionality with four additional NMOS transistors and one capacitor to enable in-memory computing while preserving all conventional SRAM operations.
2. The proposed 10T1C-SRAM cell can perform both in-memory AND and XNOR operations in a single cycle through optimized control signal sequencing, significantly enhancing computational efficiency for AI applications.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:03:46 UTC from IEEE Xplore. Restrictions apply.

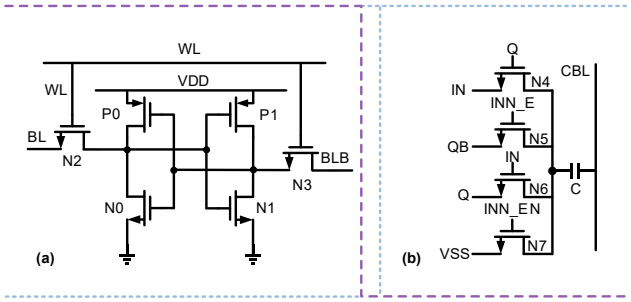


Fig. 2. (a) Traditional 6T Unit (b) Computing Module

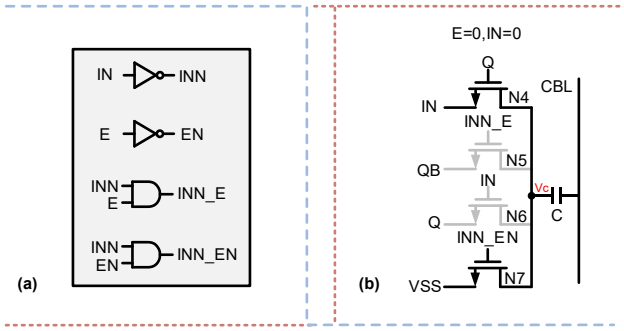


Fig. 3. (a) Ctrl Module (b) When E=0 and IN=0, the cell is in storage mode.

3. The 10T1C-SRAM architecture supports direct in-memory 1-bit AND MAC and BNN computations, eliminating the need for data movement between memory and processing units and thereby reducing energy consumption.

II. Proposed 10T1C SRAM Cell Design

A. Computational Unit Structure

The computational unit integrates a standard 6T-SRAM memory core with an additional computing module (Fig. 2). The 6T memory core consists of cross-coupled inverters (transistors P0/N0 and P1/N1) and access transistors (N2, N3), providing conventional SRAM functionality. It stores a weight bit in complementary nodes Q (weight) and QB (weight complement). The storage nodes connect to bitlines BL and BLB via N2 and N3, respectively, under control of the word line WL.

The computing module adds four NMOS transistors (N4–N7) and one capacitor C tied to a compute bitline (CBL). Each transistor is configured as follows:

N4: Gate-Q, Source-IN, Drain-C
N5: Gate-INN_E, Source-QB, Drain-C
N6: Gate-IN, Source-Q, Drain-C
N7: Gate-INN_EN, Source-VSS, Drain-C

Here Q and QB denote the stored weight (logic 1 or 0), IN is the input, and INN_E/INN_EN are internally generated control signals. By selectively activating IN, INN_E, and INN_EN, the computing module couples the stored weight and input to the capacitor C, enabling in-memory logic computation.

B. Unit Operating Mode

The proposed 10T1C-SRAM computing-in-memory cell supports multiple configurable operating modes—including data storage, AND computation, and XNOR computation—through flexible control signaling. To realize these functions, a dedicated control module (CTRL), illustrated in Figure 3(a), has been designed. This module consists of two inverters (INV0, INV1) and two AND logic gates (AND0, AND1). Specifically, INV0 generates the complementary signal INN from the input IN, while INV1 inverts the global enable signal E to produce EN. AND0 and AND1 perform logical AND operations between INN and E, and between INN and EN, respectively, yielding the internal control signals INN_E and INN_EN. This associative control mechanism dynamically produces the required signal combinations based on the computational mode, thereby enabling precise switching of transistors within the computational unit..

In data storage mode, when the input signal IN is set to 0 and the master control signal E remains at 0, the internal node INN presents a high logic level ("1") through inversion, while EN also becomes "1" due to the inversion of E. Under these conditions, the derived control signals set INN_E to "0" and INN_EN to "1". This signal combination forces transistor N5 into the off state while turning on N7 (as shown in Figure 3(b)). Consequently, all transistors within the computational cell remain disabled, effectively isolating the computational cell. The cell then operates solely based on the standard 6T-SRAM structure, exhibiting identical functional behavior to a classic 6T-SRAM memory cell. In this state, the circuit reliably executes conventional SRAM functions—including weight data read, write, and hold operations—without any interference from the computational cell, ensuring the stability and integrity of storage operations.

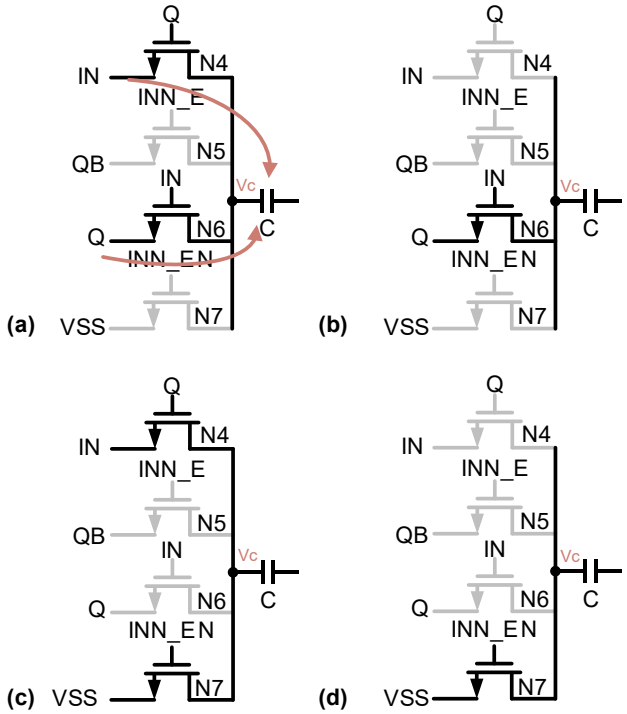


Fig. 4. When E=0, the cell switches to the in-memory AND mode. (a) Input=1, Weight=1. Result=1.(b) Input=1, Weight=0. Result=0. (c) Input=0, Weight=1. Result=0.(d) Input=0, Weight=0. Result=0.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:03:46 UTC from IEEE Xplore. Restrictions apply.

In the in-memory AND computation mode, the logic levels of the input and weight signals are defined as follows: IN=1 indicates Input=1, IN=0 indicates Input=0; Q=1 indicates Weight=1, Q=0 indicates Weight=0.Before executing the AND calculation, a reset operation is required: Set E=0 and IN=0 (at this point, N6 is cutoff), resulting in INN_E=0 (N5 cutoff) and INN_EN=1 (N7 conduction). At this moment, the first terminal of capacitor C is connected to VSS via N7 and reset to the VSS level. It should be noted that nodes Q and QB retain the values stored after the previous calculation, which could be Q=1, QB=0, or Q=0, QB=1. If Q=1 and QB=0, N4 conducts. The first terminal of C is also connected to IN (currently 0) via N4 and is similarly pulled down to VSS. If Q=0 and QB=1, N4 turns off. The first terminal of C is reset to VSS solely via N7.

During in-memory AND calculations, the control signal maintains E=0. As shown in Figure 4(a), Only when IN=1 and Q=1 is the first terminal of C charged to a high level, yielding an AND result of "1"; if IN=0 or Q=0, the first terminal of C remains at VSS, resulting in an AND value of "0".

In the in-memory XNOR computation mode, the logical encoding of inputs and weights is defined as follows: IN=1 denotes Input=1, IN=0 denotes Input=-1; Q=1 denotes Weight=1, Q=0 denotes Weight=-1.

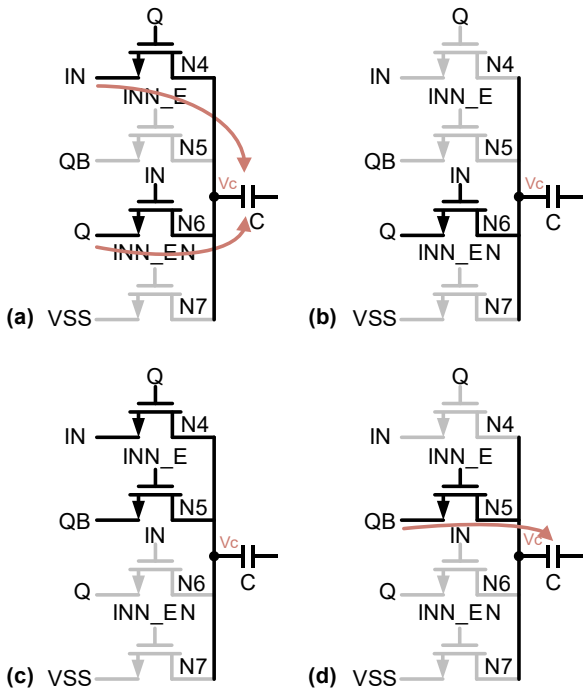


Fig. 5. When E=1, the cell operates in the in-memory XNOR mode. (a) Input=1, Weight=1. Result=1. (b) Input=1, Weight=-1. Result=-1. (c) Input=-1, Weight=1. Result=-1.(d) Input=-1, Weight=-1. Result=1.

Similar to the AND computation mode, a reset is required before executing the XNOR operation: set E=0, IN=0 (N6 cutoff), at which point INN_E=0 (N5 cutoff) and INN_EN=1 (N7 conduction). Terminal 1 of capacitor C is connected to VSS via N7, thereby resetting it to the VSS level. At this point, storage nodes Q and QB retain the results from the previous computation, potentially being Q=1, QB=0 or Q=0, QB=1. If Q=1 and QB=0, N4 conducts, and the first terminal of C is simultaneously connected to IN (currently 0) via N4, providing auxiliary pull-down. If Q=0 and QB=1, N4 is cutoff, and the first terminal of C is reset solely via N7.

During XNOR computation, the control signal sets E=1. As shown in Figure 5(a),When the input equals the weight (IN = Q), the first terminal of C charges to a high level, yielding an XNOR result of 1. If IN ≠ Q, the first terminal of C remains at VSS, resulting in an XNOR output of -1.

In the three input weight combinations shown in Figures 5(b), (c), and (d): When Input=1, Weight=-1 (IN=1, Q=0, INN=0, QB=1; INN_E=0, INN_EN=0), N4, N5, and N7 turn off while N6 turns on. The first terminal of C remains at VSS, and the XNOR output is -1. When Input=-1, Weight=1 (IN=0, Q=1, INN=1, QB=0; INN_E=1, INN_EN=0), N4 and N5 conduct while N6 and N7 turn off. Terminal 1 of C remains at VSS, and the XNOR output is -1. When Input=-1 and Weight=-1 (IN=0, Q=0, INN=1, QB=1; INN_E=1, INN_EN=0), N5 turns on, N4, N6, and N7 turn off, the first terminal of C charges to VDD' through N5, and the XNOR output is 1.

C. *Memory-Computing Array Circuit Structure*

As shown in Figure 6, the compute-in-memory circuit based on 10T1C-SRAM cells consists of a 4×4 array and peripheral control circuitry. The array contains 16 cells (as in Example 1), with each row sharing a WL and a control module (CTRL), and each column sharing a BL, its complement (BLB), and a CBL.

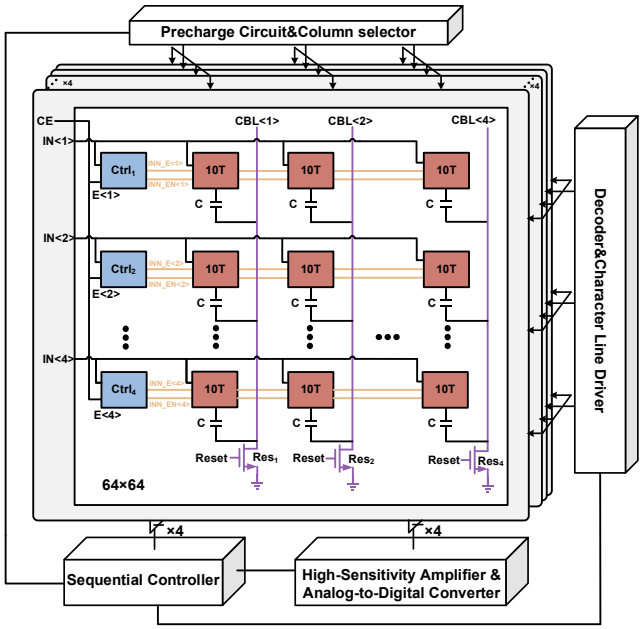


Fig. 6. Circuit Diagram of a Compute-in-Memory Circuit Based on a 10T1C-SRAM In-Memory Computing Cell

The design includes four CTRL modules and four CBLs. All CTRL enable inputs (E) connect to the global signal CE, while their input signals (IN) are independent. Each CBL is connected to VSS through an NMOS reset transistor, with all reset gates driven by a common Reset signal.

Each row uses dedicated word lines (WL_1–WL_4) and control modules ($CTRL_1$–$CTRL_4$), each with E tied to CE and IN individually controlled. Each column shares bit lines (BL_1–BL_4),

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:03:46 UTC from IEEE Xplore. Restrictions apply.

complementary lines (BLB_1–BLB_4), and compute lines (CBL_1–CBL_4). Each CBL is pulled down via a corresponding reset transistor (RES_1–RES_4), whose gate, drain, and source connect to Reset, CBL, and VSS, respectively.

The peripheral control circuit manages operation and includes a row decoder, word-line driver, precharge circuit, column selector, sense amplifier, ADC, and timing controller. The decoder addresses rows; the driver activates word lines; the precharge circuit prepares nodes; the selector picks columns; the sense amplifier reads weight data; the ADC quantizes CBL voltages; and the timing controller synchronizes all signals. Alternate peripheral implementations are acceptable if they support the in-memory architecture.

D. Array Computing

The 10T1C-SRAM in-memory computing cell supports five operating modes: Data Storage, In-Memory AND, In-Memory XNOR, AND MAC, and In-Memory BNN. For the first three modes, the peripheral control circuit selects the target unit and suppresses same-column interference using previously established control methods[9].

In the in-memory AND MAC computation mode, the array performs parallel multiply-accumulate (MAC) operations between 1-bit inputs and multi-bit stored weights. Initialization begins by setting CE = 0, all IN‹m› = 0, and Reset = 1. This discharges all computation bit lines (CBLs) and cell capacitors to VSS, clearing any residual charge.

As shown in Figure 7, during computation, CE remains 0 to keep cells in AND mode, while Reset is set to 0, allowing CBLs to enter a high-impedance state. Row input signals are applied independently. Each cell performs an AND operation between its weight Q and input IN: if the result is 1, its capacitor charges to VDD'; otherwise, it remains at VSS.

Accumulation uses charge sharing: all capacitors in a column share the same CBL, so the voltage VCBL is proportional to the number n of charged cells: VCBL = n × (VDD' / 4). For example, in a 4-cell column, if only one cell outputs 1, VCBL reaches VDD'/4, representing a MAC result of 1. An on-chip ADC directly quantizes VCBL, enabling data-free accumulation and improving parallelism and energy efficiency.

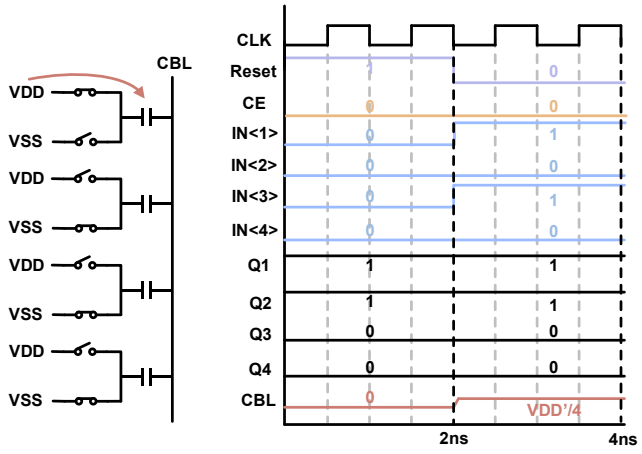


Fig. 7. Schematic of 1b-AND MAC computation in a column of 10T1C-SRAM cells within a compute-in-memory array (4×4 examples).

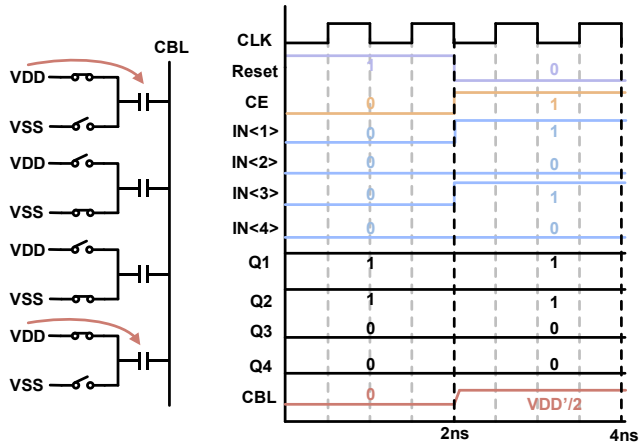


Fig. 8. Schematic Diagram of In-Memory Computing Circuit for In-Memory BNN Computation

Multi-column arrays extend this mechanism: each column acts as an accumulation channel. A 4-column array computes four parallel 1b×4b MAC operations (with weights 8,4,2,1). Multiple such banks operate concurrently, supporting higher-precision operations like 4b×4b MAC for greater computational throughput.

As shown in Figure 8, in the in-memory BNN computation mode, the process comprises initialization and computation phases. During initialization, CE and all IN signals are set to 0, resetting each cell's capacitor first terminal to VSS. Simultaneously, Reset=1 activates column NMOS reset transistors, pulling CBLs and all capacitor second terminals to VSS to clear residual charge.

In the computation phase, CE=1 configures cells for XNOR operation, while Reset=0 places CBLs in high-impedance state. Each column's four cells perform parallel XNOR operations between stored weights Q and inputs IN: matching values yield "1", charging C to VDD′, mismatches yield "-1", leaving C at VSS.

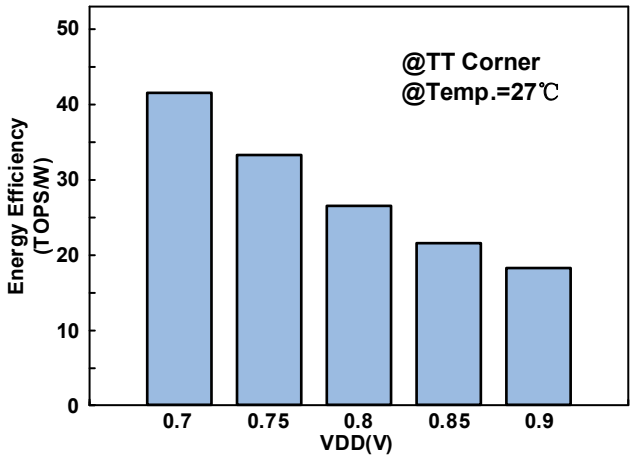


Fig. 9. Energy efficiency chart under different VDD values

The shared CBL voltage VCBL reflects the number n of "1" outputs: VCBL = (n/4)·VDD′, corresponding to an algebraic sum of 2n - 4. For example, VCBL = VDD′/2 indicates two "1" and two "-1" outputs, summing to 0. An on-chip ADC quantizes

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:03:46 UTC from IEEE Xplore. Restrictions apply.

this voltage to directly produce the BNN result, eliminating data movement and significantly improving inference parallelism and energy efficiency.

III. Simulation Results and Analysis

This design is implemented in a 16Kb SRAM array using a 28nm CMOS process. Figure 9 shows the peak energy efficiency of the AND CIM. At a supply voltage of 0.7V, the energy efficiency is 41.8 TOPS/W. At 0.9V, the energy efficiency is 18.3 TOPS/W. Compared to other architectures, this design can achieve both MAC computations while also accommodating CNN calculations, significantly improving usability and energy efficiency. Table I compares the previous MAC computation performance with the performance of this work. Clearly, this design significantly optimizes energy consumption and functionality. Overall, the structure proposed in this paper will have better flexibility than existing structures[10].

TABLE I. COMPARISON WITH PRIOR WORKS

	JSSC'17[11]	*JSSC'22[12]*	*ISSCC'22[13]*	*JSSC'23[14]*	*This Work*
Technology (nm)	130	65	28	22	28
Bitcell Density (Kb)	16	16	128	128	16
Cell-Type	6T	8T	6T	6T+3T1C	10T
Supply Voltage (V)	1	0.45/0.8	0.65-0.9	0.7-1.1	0.7-0.9
Input/Weight (bit)	5/1	1/1	8/8	8/8	4/1
Output(bit)	1	1-5	22	7	4
Frequency of operation(Mhz)	50	200	5	N/A	250
Energy Efficiency (TOPS/W)	11.51	15.8-490	21.2-27.75	15.5-32.2	18.3-41.8

IV. Conclusion

This article introduces a 16Kb 8T SRAM compute-in-memory (CIM) macro implemented based on 28nm CMOS technology. The design integrates data storage, AND logic computation, XNOR logic computation, in-memory AND multiply-accumulate operations, and convolutional neural network (CNN) computation functionalities, providing high application flexibility. This macro supports 1-bit weight multiply-accumulate (MAC) operations and CNN computations, achieving an energy efficiency of 18.3-41.8 TOPS/W within a working voltage range of 0.7-0.9 V. Its excellent performance in energy efficiency fully demonstrates the macro's potential in artificial intelligence edge computing applications and has positive significance for promoting the practical development of SRAM-CIM technology.

References

[1] S. Aga, S. Narayanan, K. Roy, and A. Raghunathan, “Compute caches: Leveraging existing cache hierarchies for neural network acceleration,” in *Proc. Design Automation Conf. (DAC)*, 2017, pp. 1–6.

[2] A. Shafiee, A. Nag, N. Muralimanohar, et al., “ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars,” in *Proc. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 14–26.

[3] P. Chi, S. Li, C. Xu, et al., “PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory,” in *Proc. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 27–39.

[4] J. Zhang, M. Kang, and M. Annavaram, “In-memory computation for energy-efficient neural network accelerators,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2408–2421, Oct. 2020.

[5] X. Si, H. Jiang, J. Deng, Y. Wang, and H. Yang, “A twin-8T SRAM computation-in-memory unit-macro for energy-efficient CNN inference,” *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 189–202, Jan. 2020.

[6] A. Biswas and A. Chandrakasan, “CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks,” *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, Jan. 2019.

[7] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, “XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks,” *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.

[8] Y. Kim, S. Lee, J. Park, et al., “A 16-Kb computing-in-memory SRAM with reconfigurable read capability for binary neural networks,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2021, pp. 252–253.

[9] H. Jia, J. Zhao, and Y. Xie, “Efficient in-memory computing with spintronics: Opportunities and challenges,” *Proc. IEEE*, vol. 108, no. 8, pp. 1322–1342, Aug. 2020.

[10] T. Yang, Y. Wang, and H. Yang, “Design techniques for SRAM-based in-memory computing macros in AI accelerators,” in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, 2019, pp. 57–60.

[11] J. Zhang, Z. Wang, and N. Verma, "In-Memory Computation of a Machine-Learning Classifier in a Standard 6T SRAM Array," *IEEE Journal of Solid-State Circuits,* vol. 52, no. 4, pp. 915-924, 2017.

[12] C. Yu, T. Yoo, K. T. C. Chai, T. T.-H. Kim, and B. Kim, "A 65-nm 8T SRAM Compute-in-Memory Macro With Column ADCs for Processing Neural Networks," *IEEE Journal of Solid-State Circuits,* vol. 57, no. 11, pp. 3466-3476, 2022.

[13] C. Yu, T. Yoo, K. T. C. Chai, T. T.-H. Kim, and B. Kim, "A 65-nm 8T SRAM Compute-in-Memory Macro With Column ADCs for Processing Neural Networks," *IEEE Journal of Solid-State Circuits,* vol. 57, no. 11, pp. 3466-3476, 2022.

[14] P.-C. Wu *et al.*, "A 28nm 1Mb Time-Domain Computing-in-Memory 6T-SRAM Macro with a 6.6ns Latency, 1241GOPS and 37.01TOPS/W for 8b-MAC Operations for Edge-AI Devices," presented at the 2022 IEEE International Solid- State Circuits Conference (ISSCC), 2022.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:03:46 UTC from IEEE Xplore. Restrictions apply.